

UNIVERSITÀ DI PARMA
Dipartimento di Ingegneria e Architettura

Symmetric Key Establishment

Luca Veltri

(mail.to: luca.veltri@unipr.it)

Course of Cybersecurity, 2025/2026

Key management

- In a secured communication, users must setup the details of the cryptography
  - **in some cases this may require sharing a symmetric key**
  - **in others it may require obtaining the other party's public key**
- In both cases, an issue is how to securely distribute these keys
  - **in case of symmetric key**
    - must be exchanged over a secure communication channel
      - both confidentiality and data authentication must be guaranteed
  - **in case of public key**
    - keys can be exchanged authenticated over an insecure communications channel
      - only message authentication must be guaranteed
        - » the exchange is less troublesome
        - » for example, by using digital certificates
- Often secure systems fail due to a break in the key distribution

Long-term vs Short-term Keys

- Cryptographic system and protocols usually deal with two types of keys with two different lifetimes:
  - **long-term keys**
    - also referred to as long-term secrets
    - can be used for
      - peer authentication
      - securing key exchanges
  - **short-term keys**
    - also called as session keys
    - used to secure a communication
      - data confidentiality
      - data authentication and integrity
    - used only for a limited interval of time
    - securely setup using long-term keys through a proper KE protocol

(Secret) Key establishment

- Secret key establishment is a process or protocol whereby a shared secret becomes available to two or more parties, for subsequent cryptographic use
- Two approaches are possible:
  - **key pre-distribution**
    - keys are completely determined “a priori” by initial keying material
      - used mainly for long-term secret keys
  - **dynamic key establishment (or key exchange)**
    - those schemes whereby the key established by a pair (or group) of users varies on subsequent executions
    - used also for key updates
    - subdivided into (see later):
      - key transport
      - key agreement

Key establishment (cont.)

- Dynamic key establishment may be broadly subdivided into:
  - **key transport**
    - A key transport protocol or mechanism is a key establishment technique where one party creates or otherwise obtains a secret value, and securely transfers it to the other(s)
  - **key agreement**
    - A key agreement protocol or mechanism is a key establishment technique in which a shared secret is derived by two (or more) parties as a function of information contributed by, or associated with, each of these, (ideally) such that no party can predetermine the resulting value
- Additional variations exist, including various forms of key derivation

Use of trusted servers

- Some key establishment protocols involve a centralized or trusted party, for either or both:
  - **initial system setup**
  - **on-line actions (involving real-time participation)**
- This party is referred to by a variety of names depending on the role played
  - **generically called as trusted third party or trusted server**
- E.g.
  - **key distribution center (KDC)**
  - **authentication server**
  - **certification authority (CA)**

Key Establishment Security Properties

- Implicit Key Authentication (IKA)
  - **a party A has implicit key authentication of B if A is assured that no party other than the intended peer B can learn the established session key**
    - about exclusivity of key knowledge
      - only B could know the key
    - does not guarantee that B actually computed the key
      - passive assurance
    - neither guarantee that the peer actually *is* online right now
- Key Confirmation (KC)
  - **a party A has key confirmation from B if A is assured that B actually possesses the correct session key**
    - active assurance peer has the key
      - proof of possession of key
  - **typically achieved via MAC or encrypted challenge/response**

Key Establishment Security Properties (cont.)

- Explicit Key Authentication (EKA)
  - **a party A has explicit key authentication of B if:**
    - A has implicit key authentication of B, and
    - A has key confirmation from B
  - **A is assured that:**
    - only B can know the key
    - and B actually does know it
  - **combines implicit authentication + key confirmation**
- Peer Entity Authentication (PEA)
  - **a party A has peer entity authentication of B if A is assured that:**
    - the communicating entity is indeed B, and
    - B is actively participating in the protocol at the current time liveness/freshness
  - **that is: identity verification (who) + freshness (now, not replay)**
    - this is really B, and B is live right now
  - **often achieved via nonces, timestamps, challenge-response**

Key Establishment Security Properties (cont.)

- Differences:
  - **Implicit key authentication**
    - ensures key secrecy with respect to others
      - only intended peer can know key
  - **Key confirmation**
    - ensures peer actually computed key
      - peer actually knows the key
  - **Explicit key authentication**
    - combines both the above
  - **Peer entity authentication**
    - adds identity + liveness guarantees
      - correct identity + active participation
- In addition:
  - **Key freshness**
    - assurance that the exchanged key is new

Unknown Key-Share attack

- Unknown Key-Share attack (UKS)
  - **the key is correct, but the association between key and identity is wrong**
  - **e.g.**
    - A **shares the key with** B, but A **believes the key is shared with** B, while B **believes it is shared with** M
      - A thinks: "I share key K with B" (this part is actually true)
      - B thinks: "I share key K with M" (this is wrong)
    - the key is **not shared with** M
      - M **never learns the key**
  - **this is still an attack because even though confidentiality is preserved (**M **doesn't know K)**
  - **the attack breaks:**
    - **identity agreement**
    - **authentication semantics**

Authenticated Key Exchange (AKE)

- A protocol is an Authenticated Key Exchange (AKE) if it allows two (or more) parties to
  - **securely establish a shared session key, and**
  - **be assured of the identity of the peer(s) with whom the key is shared**
- AKE usually includes:
  - **explicit key authentication**
    - assurance about peer's involvement and key possession
  - **peer entity authentication**
    - identity agreement and active participation
  - **freshness**
    - the key is new (not replayed)

Forward secrecy and known-key attacks

- It is important to consider the potential impact of compromise of various types of keying material
  - **even if such compromise is not normally expected**
- In particular, the effect of the following is often considered:
  - **compromise of past session (established) keys**
    - a protocol is said to be "resistant to a known-key attack" if compromise of past session keys doesn't allow an adversary to compromise future session key exchanges
  - **compromise of long-term secret (symmetric or asymmetric) keys**
    - A protocol is said to have "(perfect) forward secrecy" if compromise of long-term keys does not compromise past session keys
      - for example, Diffie-Hellman key agreement, in some cases, may provide forward secrecy

Key transport and derivation

Point-to-point key transport and key derivation

- Sometimes the two parties A and B already share “a priori” a (long-term) symmetric key K_{AB} and they want to exchange a new short term key K_S
  - **the long-term key must be initially exchanged over a secure channel or resulting from a key pre-distribution mechanism**
- Possible techniques:
  - **key transport based on symmetric encryption**
  - **key derivation based on non-reversible functions**
- Variants:
  - **one pass**
  - **challenge-response**

Key transport based on symmetric encryption

- Key transport with one pass:

$$A \rightarrow B: \quad E_{Kab}(K_S)$$

  - **both A and B obtain implicit key authentication**
  - **susceptible to known-key attacks through replay attack**

- Additional optional fields might be transferred in the encrypted portion:

$$A \rightarrow B: \quad E_{Kab}(K_S, t_A^*, B^*)$$

  - **field containing redundancy provides explicit key authentication to B and facilitates message modification detection**
  - **timestamp (or sequence number) provides a sort of freshness guarantee to B**
    - avoids replay attacks
  - **a destination identifier prevents message replay back on A**
    - if a timestamp is present it provides also entity authentication to B

Key transport (cont.)

- Key transport with challenge-response:
  - **If anti-replay, explicit key authentication, and entity authentication are desired for B but reliance on timestamps added by A is not, a random value or sequence number n_B may be used**
  - **the cost is an additional message**

$$A \leftarrow B: \quad n_B$$

$$A \rightarrow B: \quad E_{Kab}(K_S, n_B, B^*)$$

- If it is required that the session key K_S be a function of inputs from both parties and mutual authentication:

$$A \leftarrow B: \quad n_B$$

$$A \rightarrow B: \quad E_{Kab}(k_1, n_A, n_B, B^*)$$

$$A \leftarrow B: \quad E_{Kab}(k_2, n_B, n_A, A^*)$$

  - **$K_S = f(k_1, k_2)$**

Key transport (cont.)

- Vulnerabilities:
  - **previous exchange does not offer forward secrecy**
    - they fail if the long-term key K_{ab} is compromised
      - they may be inappropriate for many applications
- Note:
  - **Authentication protocols which employ encryption, including the above key exchange schemes, may require that the encryption function includes data integrity mechanism to detect message modification**
    - for example using an Authenticated Encryption (AE) algorithm

Key derivation

- Key exchange based on key derivation with one pass
  - **the session key is derived from a per-session random input provided by one party**

$$A \rightarrow B: \quad r_A$$

  - **the session key is computed as $K_S = f_{Kab}(r_A)$**
    - where *f()* can be either a $E_K()$ or $MAC_K()$
  - **provides to both A and B implicit key authentication**
  - **susceptible to known-key attacks through replay attack**

- The random number r_A may be replaced by other time-variant parameters
  - **e.g. a timestamp t_A, provides an implicit key freshness property**
    - avoids replay attack to *B*
    - avoids *A* can force a given key *X*

Key derivation (cont.)

- A keyed one-way function like $MAC_K()$ is usually preferred
  - **respect an encryption function $E_k()$**
  - **A cannot control the value of K_S**

- The simple key derivation scheme can be further extended with two or three passes in order to provide:
  - **contribution of B to the creation of K_S**
  - **mutual entity authentication**
  - **example, AKEP2**

Example of authenticated key derivation protocol

- Authenticated Key Exchange Protocol AKEP2
  - **provides mutual entity authentication, key freshness guarantee, and implicit key authentication**
  - **does not provide forward secrecy**

- Protocol:
  - **A and B share long-term symmetric keys K_{ab}**

$$A \rightarrow B: \quad r_A$$
$$A \leftarrow B: \quad X_B = \{B, A, r_A, r_B\}, MAC_{Kab}(X_B)$$
$$A \rightarrow B: \quad X_A = \{A, r_B\}, MAC_{Kab}(X_A)$$

  - **$K_S = f_{K'}(r_B)$**
    - where $f_K()$ is a keyed one-way function or encryption function
    - K' is the key used to derive Ks, usually obtained from K_{ab}

Key transport based on public-key encryption

- Also referred to as Key Encapsulation Mechanisms (KEM)
- One party may choose a symmetric key and transfer it to a second, using that party's encryption public key

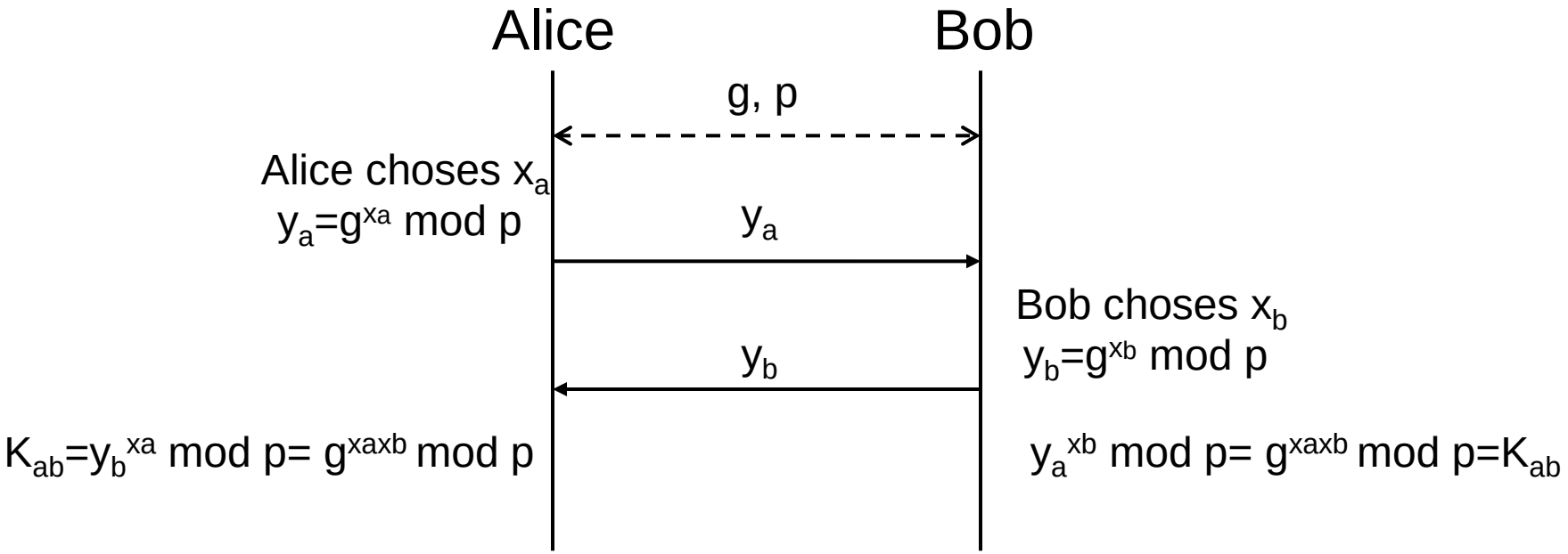
$$\mathbf{A \rightarrow B :\ \ \{ K_S \} KU_B}$$

  - **this provides (implicit) key authentication to the originator (A)**
    - only the intended recipient has the private key allowing decryption
    - the originator (A) obtains neither entity authentication nor key confirmation
  - **the second party (B) has no assurances regarding the source of the key and the timeliness**
    - even if A sends to B: $\{K_S, A, T_A\} KU_B$, since KU_B is public
  - **such additional assurances may be obtained through use of further techniques including:**
    - additional messages, with public-key encryption
      - using challenge-response like key transport based on symmetric key
    - using digital signature

Key agreement
using asymmetric cryptography

Diffie-Hellman key exchange

- The first publicly known public-key agreement protocol was the Diffie-Hellman exponential key exchange



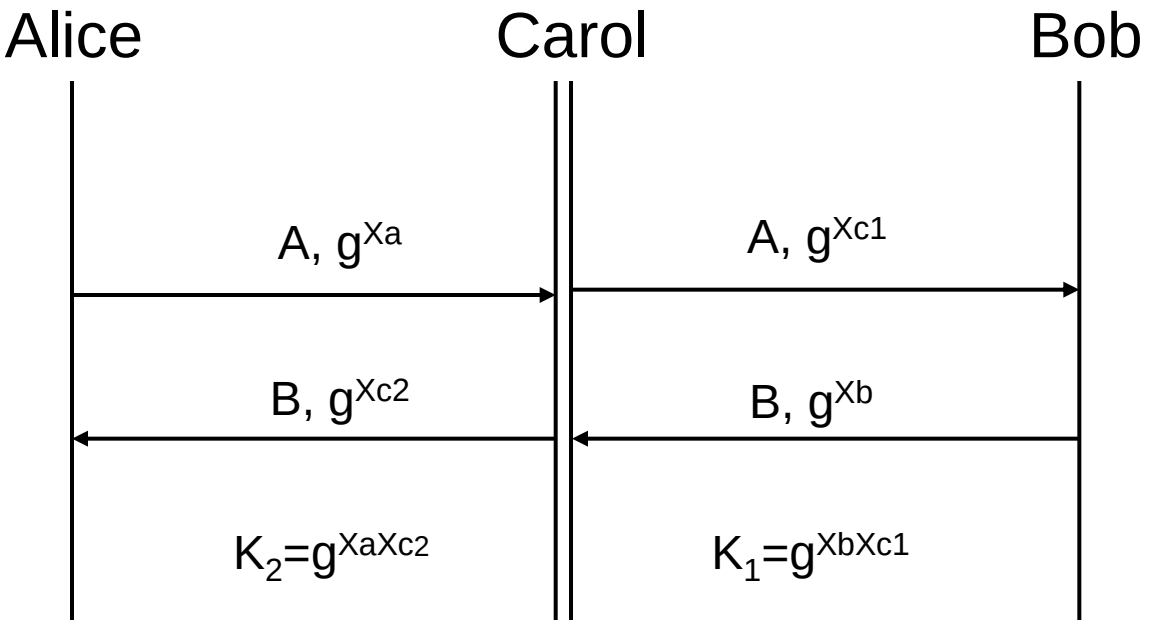
DH modes

- Different ways of using DH:
  - **Fixed Diffie-Hellman**
    - each party already has the public part (DH public key) of the other entity
  - **Anonymous Diffie-Hellman**
    - DH exchange
    - susceptible to Man-in-the-Middle attacks
  - **Ephemeral Diffie-Hellman (EDH)**
    - authenticated DH exchange

- Elliptic Curve variant:
  - **Elliptic Curve DH (ECDH) uses elliptic curves instead of the multiplicative group of integers modulo *p***

Diffie-Hellman Vulnerability (MITM attack)

- Anonymous DH key exchange does not provide authentication of the parties, and is thus vulnerable to Man-in-the-middle attacks
  - **peers don't verify the identity of the other peer**
  - **a third party C may run two separate exchanges with A and B, convincing the two peers that the exchange ended successfully between them**

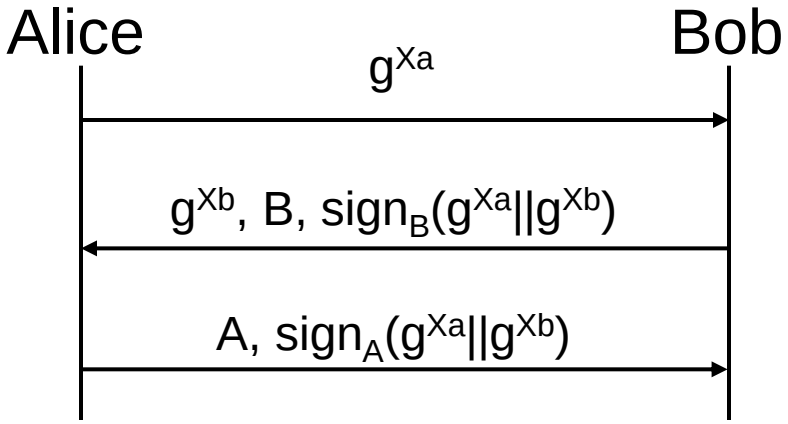


Authenticated DH Exchange

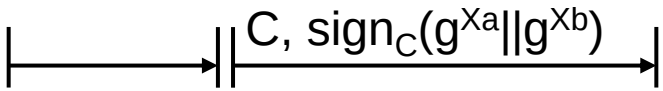
- Different schemes and protocols have been developed to provide authenticated DH key agreement to prevent man-in-the-middle and related attacks
- These methods generally use:
  - **Public/private key pairs**
  - **Shared secrets**

Authenticated DH Exchange (cont.)

- The simplest way to authenticate the DH exchange could be to sign the DH exponentials



  - **It does not guarantee implicit key authentication nor confirmation**
    - the authenticated entity may be different from the peer that exchanged the secret key
      - a third party C can convince B that the exchange was run with C, by simply replacing the third message with:



      - this attack does not result in a breach of secrecy of the key

Authenticated DH Exchange (cont.)

- Solution: uses signature and Enc
  - **variant of Station-to-Station (STS) protocol**

$$A \rightarrow B: \quad g^{Xa}$$

$$A \leftarrow B: \quad g^{Xb}, E_{Ks}(B \,||\, Sign_B(g^{Xa} \,||\, g^{Xb}))$$

$$A \rightarrow B: \quad E_{Ks}(A \,||\, Sign_A(g^{Xa} \,||\, g^{Xb}))$$

- Solution: uses signature and MAC
  - **SIGMA Protocol:**
    - signatures authenticate the DH exponentials
    - MACs bind the key to identities
      - MAC is performed with a key K_m derived by $K_S=g^{XaXb}$

$$A \rightarrow B: \quad g^{Xa}$$

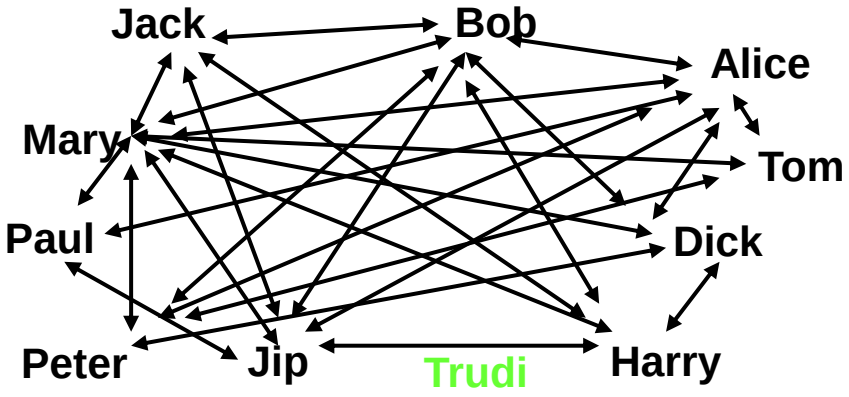
$$A \leftarrow B: \quad B, g^{Xb}, Sign_B(g^{Xa} \,||\, g^{Xb}), MAC_{Km}(B)$$

$$A \rightarrow B: \quad A, Sign_A(g^{Xa} \,||\, g^{Xb}), MAC_{Km}(A)$$

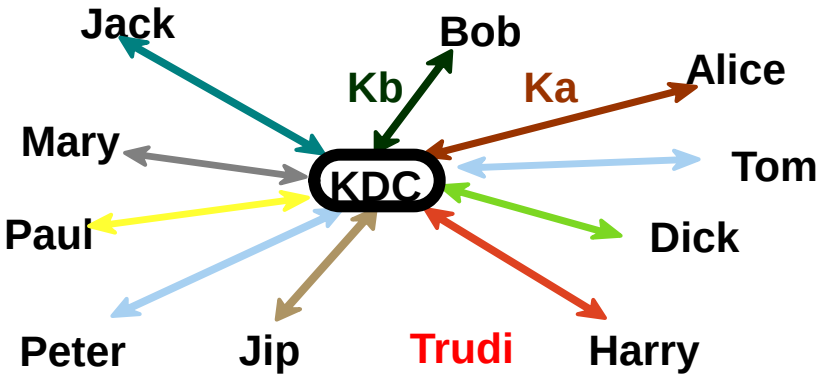
Server-based Key Distribution

Key Distribution using a Trusted Third Party

- With *N* nodes, each node must authenticate each other.. *N*-1 keys maintained by each node

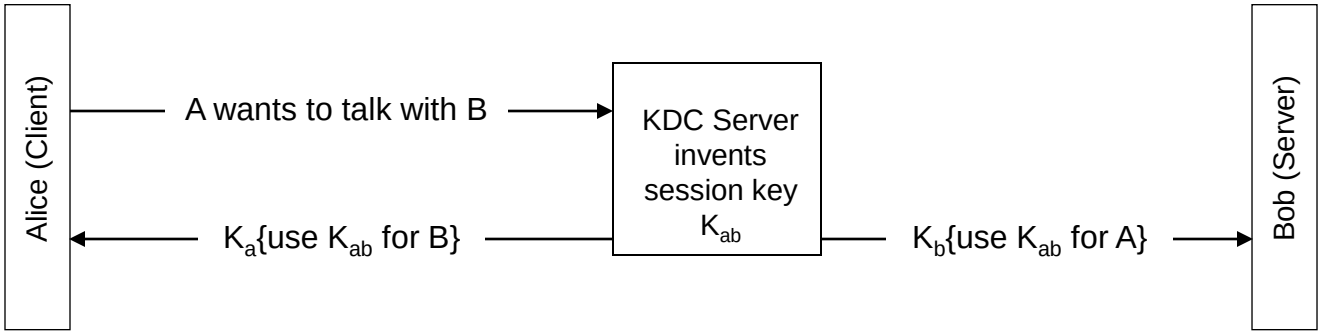


- Possible solution: Trusted intermediate party
  - **Key Distribution Center (KDC)**
  - **similar to CAs for public-key cryptography**



Key Distribution using a KDC

- Key Distribution in theory (Client-Server)



- Key Distribution in practice (Client-Server)
  - **e.g. used by Kerberos protocol**

